ENGENHARIA DE COMPUTAÇÃO · QUALIDADE DE SOFTWARE

Projeto Final de Qualidade de Software

Plano, projeto de testes, automação e relatório de defeitos aplicados ao Auditor de Conformidade FIWARE

Curso	Engenharia de Computação — 9º semestre
Disciplina	Qualidade de Software
Professor	Prof. Dr. Carlos Noriega
Sistema avaliado	Auditor de Conformidade FIWARE (TCC DKRT) — backend Python/FastAPI e frontend Angular
Grupo	[preencher com os nomes dos integrantes]
Repositório	TCC-CEFSA-DKRT/TCC_DKRT — branch improve/qualidade-software-pfc
Data	Novembro de 2026

Entrega Única

Este documento consolida o **Módulo A** (plano de testes, APF e COCOMO II), o **Módulo B** (testes funcionais caixa-preta e análise de usabilidade/IHC) e o **Módulo C** (automação em pytest, testes de carga e relatório de defeitos e cobertura). Os scripts executáveis estão no repositório; a apresentação em PowerPoint acompanha este arquivo.

Sumário

Resumo executivo 3
1. Escopo: o que testar e o que não testar 4
Módulo A — Plano de Testes e Estimativa de Esforço 5
A.1 Plano de Testes (ISO/IEC 29119-3) 5
A.2 Estimativa por Análise de Pontos de Função (APF) 6
A.3 Modelo COCOMO II e equipe de QA 8
Módulo B — Projeto de Testes Funcionais e Usabilidade 10
B.1 Técnicas aplicadas e rotinas escolhidas 10
B.2 Classes de equivalência: StructuralValidator (RF-01) 10
B.3 Tabela de decisão: derivação do status (RF-02) 11
B.4 Tabela de decisão: interpretação de comando (RF-03) 11
B.5 Entradas da API: classes e valores-limite (RF-03 a RF-07) 11
B.6 Roteiro e especificação de casos de teste 12
B.7 Análise de usabilidade e IHC 13
Módulo C — Automação, Carga e Relatório de Qualidade 15
C.1 Arquitetura da suíte 15
C.2 Resultados da execução 15
C.3 Cobertura de código 16
C.4 Testes de carga e desempenho 16
C.5 Eficácia da suíte: verificação por mutação manual 18
C.6 Matriz de rastreabilidade 18
C.7 Relatório de defeitos 19
Conclusão 20
Referências 21
Anexo A — Roteiro completo dos casos de teste 22

Resumo executivo

O trabalho foi feito sobre o próprio TCC, e não sobre um exemplo à parte. Antes de escrever qualquer teste mediu-se a situação de partida (480 testes passando, 86,6% de cobertura), e a escolha do que testar seguiu o risco real do sistema: o que mais pode falhar é decidir errado se uma entidade está em conformidade, aceitar entradas inválidas na API e falhar em silêncio nas funções Lambda de produção.

Indicador	Resultado
Tamanho funcional (APF)	118 PF não ajustados · 134,5 PF ajustados (VAF 1,14) · 27 funções
Esforço total (COCOMO II)	13,0 pessoas-mês em 8,4 meses (≈ 1,6 pessoas em média)
Esforço de QA estimado	3,2 pessoas-mês ≈ 492 horas; 1,0 testador em tempo integral por 3,3 meses
Casos de teste formais	125 casos rastreados a 10 requisitos (123 em pytest, 2 em Locust)
Resultado da execução	113 passam · 12 expõem defeitos conhecidos (xfail estrito) · 0 falhas inesperadas
Suíte completa do projeto	592 testes passam, 1 pulado, 12 xfail; antes eram 480 passando
Cobertura de instruções	86,6% → 88,7% (+63 instruções); os 3 handlers Lambda de produção saíram de 0% para 100%
Eficácia da suíte	6 de 6 mutantes introduzidos manualmente foram detectados
Defeitos registrados	10 (6 na API e no domínio, confirmados por execução; 4 na interface, por inspeção) + 1 observação de capacidade
Carga nominal (100 usuários)	0% de falhas · p95 = 150 ms · 268 req/s
Estresse (500 usuários)	0% de falhas · p95 = 1,8 s · saturação em ≈ 280 req/s com 1 worker

> **O achado mais importante**
>
> Os testes caixa-preta encontraram falhas de validação que a suíte existente não detectava: paginação sem limites (`page_size=1000000` é aceito), erro 500 em `/audits/summary-over-time` com tipo vazio e o validador estrutural aceitando booleano como inteiro e data sem hora como `date-time`. Cada um está reproduzido por um caso de teste marcado como falha esperada.

1. Escopo: o que testar e o que não testar

O enunciado pede técnicas específicas, mas a escolha das rotinas e dos tipos de teste deve responder à pergunta “o que realmente faz sentido para esta aplicação?”. A tabela abaixo registra cada tipo de teste considerado e a decisão tomada.

Tipo de teste	Decisão	Justificativa
Funcional caixa-preta do núcleo (validador e status de conformidade)	SIM	É o produto: um erro aqui gera falso “conforme” ou falso “não conforme”. Entradas numéricas e enums se prestam bem a classes de equivalência e valor-limite.
Validação de entrada da API REST (borda HTTP)	SIM	Única superfície exposta ao usuário e ao frontend. Cobre RF-03 a RF-07 com decisões objetivas (aceita/rejeita).
Interpretação de comando em linguagem natural	SIM	Tem regras de decisão explícitas (intenção × confiança × broker). O SLM é simulado com mock, pois o foco é a regra de negócio, não o modelo.
Caixa-branca dos handlers Lambda (worker, DLQ, API)	SIM	Caminho de produção (ADR-0017) que estava com 0% de cobertura. Falhar aqui perde ou duplica auditorias.
Carga e estresse da API	SIM	Auditorias em lote e produção em Lambda atrás de ALB; o enunciado exige reportar tempo de resposta e taxa de erro.
Usabilidade / IHC	SIM	São 7 telas, incluindo um assistente de linguagem natural que depende de confiança do usuário (“nada executa sem confirmação”).
Integração com Orion, Postgres/pgvector e Ollama	JÁ EXISTE	O projeto tem testes `integration` e `golden` que exigem infraestrutura ativa. Não foram duplicados; seguem como parte da suíte do TCC.
Segurança (varredura de vulnerabilidades)	PARCIAL	Login, JWT e cookie já têm testes de unidade. Teste de invasão e varredura OWASP completa ficam fora de escopo.
Compatibilidade de navegadores e dispositivos	NÃO	Ferramenta técnica de uso interno com navegador atual; custo alto, benefício baixo para o TCC.
Acessibilidade formal (WCAG)	PARCIAL	Observações pontuais no IHC (rótulos ausentes em filtros); auditoria completa fica como trabalho futuro.
Migrações Alembic e infraestrutura (Terraform)	JÁ EXISTE	Cobertos por `test_migrations.py` (integração) e `test_terraform_saas_wiring.py`.

Rotinas escolhidas para o projeto caixa-preta (o enunciado exige ao menos duas): `StructuralValidator.validate` (RF-01), `ConformanceStatus.derive` (RF-02), `InterpretAuditCommandUseCase.execute` (RF-03) e a borda HTTP — `POST /audits/interpret`, `POST /audits`, `GET /audits`, `PUT /config` e `POST /auth/login` (RF-03 a RF-07). Para a caixa-branca: `worker_handler`, `worker_dlq_handler`, `api_handler` e os ramos ainda não cobertos do validador (RF-08).

Módulo A — Plano de Testes e Estimativa de Esforço

A.1 Plano de Testes (ISO/IEC 29119-3)

Campo	Conteúdo
Identificação	PT-DKRT-01, versão 1.0, novembro de 2026
Objetivo	Verificar se o Auditor de Conformidade FIWARE decide corretamente a conformidade das entidades, valida suas entradas, processa auditorias em segundo plano sem perda e atende ao desempenho esperado.
Versão testada	Branch `main` no commit d71ca7cc (29/10/2026) mais a branch `improve/qualidade-software-pfc`
Abordagem	Baseada em risco; automação em pytest; técnicas de caixa-preta (classes de equivalência, valor-limite, tabela de decisão) e de caixa-branca (caminho básico e ramos); carga e estresse com Locust.
Responsabilidades	Líder de teste (plano, riscos, relatório); analista de teste (projeto dos casos); automatizador (scripts pytest/Locust); desenvolvedor (correção dos defeitos). No grupo, os papéis são acumulados pelos integrantes.

Itens a testar e itens não testados

Item	Requisito	Nível / técnica
Validação estrutural de entidade NGSI-v2 contra o schema	RF-01	Funcional: CE, AVL; caixa-branca (ramos)
Derivação do status de conformidade	RF-02	Funcional: tabela de decisão
Interpretação de comandos em linguagem natural	RF-03	Funcional: tabela de decisão, AVL; mocks
Iniciar auditoria assíncrona (`POST /audits`)	RF-04	Funcional da API: CE
Listagem, filtros, paginação e evolução no tempo	RF-05	Funcional da API: CE, AVL
Configuração do broker (modo self-hosted)	RF-06	Funcional da API: CE, AVL
Autenticação (modo SaaS)	RF-07	Funcional da API: CE
Processamento em segundo plano (worker e DLQ)	RF-08	Caixa-branca: caminho básico; mocks
Desempenho e robustez da API	RNF-01, RNF-02	Carga e estresse (Locust) e smoke no pytest
Interface das 7 telas	—	Usabilidade: inspeção heurística (Módulo B)

Não testados nesta entrega: integração com Orion, Postgres e Ollama reais (suítes `integration` e `golden` do projeto), qualidade das respostas do SLM, segurança ofensiva, compatibilidade de navegadores, acessibilidade formal e testes automatizados da interface (Karma). Motivos na seção 1.

Matriz de riscos técnicos

ID	Risco	Prob.	Impacto	Exposição	Mitigação nos testes
R1	Validador aceita dado inválido (falso “conforme”) ou rejeita dado válido	Média	Alto	Alta	CE e AVL em RF-01 (CT-VAL-01 a 23)
R2	Paginação sem limite degrada a API ou gera erro de banco	Alta	Alto	Alta	AVL em page e page_size (CT-PAG); defeito D-01
R3	Falha silenciosa nos handlers Lambda (perda de job, run preso)	Média	Alto	Alta	Caminho básico dos handlers (CT-WB-01 a 12)
R4	SLM indisponível ou ambíguo dispara auditoria errada	Média	Médio	Média	Tabela de decisão RF-03; limiar 0,5 por AVL

ID	Risco	Prob.	Impacto	Exposição	Mitigação nos testes
R5	Configuração de tenant inválida ou persistida sem validar	Baixa	Alto	Média	AVL do Fiware-Service e CE de URL/ServicePath (CT-CFG)
R6	Saturação sob carga	Média	Médio	Média	Locust: 100 e 500 usuários; capacidade medida
R7	Interface induz leitura errada dos dados	Média	Médio	Média	Inspeção heurística; defeitos D-07 a D-10
R8	Testes instáveis por dependência externa (Orion, rede)	Média	Médio	Média	Fakes e mocks em toda a suíte nova; sem rede

Ambiente de teste

Linux (kernel 6.18), 2 vCPU Xeon 2,8 GHz, 8 GB de RAM; Python 3.13 (o projeto exige 3.11 ou superior); pytest 9.2, pytest-asyncio 1.4, pytest-cov 7.1, FastAPI 0.162, httpx 0.28 e Locust 2.47. O gerador de carga e a API rodam na mesma máquina, com um worker do uvicorn e dependências externas substituídas por fakes em memória.

Critérios de entrada, saída, suspensão e retomada

Critério	Definição
Entrada	Código da `main` instala e importa; suíte de unidade existente verde (480 testes); ambiente reproduzível com `pip install -e ".[dev]"`.
Saída	100% dos casos de prioridade Alta executados; nenhuma falha inesperada; cobertura total ≥ 85% e sem queda; defeitos de severidade Alta documentados com recomendação; carga nominal com 0% de erro e p95 < 500 ms.
Suspensão	Falha do ambiente (dependência não instalável) ou mais de 20% dos casos falhando sem relação com defeitos conhecidos.
Retomada	Ambiente corrigido e baseline reexecutado verde.

Cronograma e distribuição do esforço

Atividade	% do esforço de QA	Horas (de 492 h)
Planejamento e análise de risco	10%	49
Projeto dos casos de teste	25%	123
Automação (pytest e Locust)	30%	148
Execução, carga e regressão	15%	74
Registro de defeitos e relatório	15%	74
Reteste e contingência	5%	25

Entregáveis: este relatório, a suíte pytest, o `locustfile.py`, os scripts de APF/COCOMO e de rastreabilidade, os CSV de resultados e a apresentação.

A.2 Estimativa por Análise de Pontos de Função (APF)

A contagem seguiu o método IFPUG, com a fronteira da aplicação cobrindo backend e frontend. As funções foram extraídas do código: rotas FastAPI, comandos da CLI, tabelas SQLAlchemy e adaptadores externos. A complexidade de cada função é calculada pelo script `scripts/qualidade/apf_cocomo.py` a partir de RET/FTR e DET, usando as matrizes padrão do IFPUG.

Tipo	Função	RET/ FTR	DET	Complex.	PF	Origem no código
ALI	Execução de auditoria (runs, relatórios, violações, recomendações)	4	48	Média	10	tables.py: audit_runs, entity_reports, violations, recommendations
ALI	Conta de tenant (broker, credenciais, login)	1	8	Baixa	7	tables.py: tenant_accounts

Tipo	Função	RET/ FTR	DET	Complex.	PF	Origem no código
ALI	Cache de schemas do catálogo	1	6	Baixa	7	schema_cache
ALI	Status de sincronização do catálogo	1	6	Baixa	7	catalog_sync_status
ALI	Base de conhecimento do RAG (chunks e embeddings)	1	6	Baixa	7	tables.py: knowledge_chunks
AIE	Entidades NGSI-v2 do Context Broker (Orion)	1	8	Baixa	5	orion_adapter.py
AIE	Catálogo e schemas Smart Data Models	2	10	Baixa	5	smart_data_models_adapter.py
AIE	Corpus de conhecimento (arquivos .md do RAG)	1	6	Baixa	5	corpus_loader.py
EE	Iniciar auditoria (API POST /audits e CLI run)	1	3	Baixa	3	routers/audits.py; cli run
EE	Autenticar usuário (login)	1	3	Baixa	3	routers/auth.py
EE	Configurar broker (PUT /config)	1	4	Baixa	3	routers/config.py
EE	Sincronizar catálogo de modelos	2	7	Média	4	cli catalog sync; catalog_sync_scheduler
EE	Sincronizar base de conhecimento	1	5	Baixa	3	cli knowledge sync
EE	Registrar tenant (administrativo)	1	6	Baixa	3	cli tenant register; admin_handler
EE	Rotacionar chaves de criptografia	1	3	Baixa	3	cli tenant rotate-keys
SE	Relatório detalhado da auditoria	1	40	Média	5	GET /audits/{id}; schemas.AuditRunResponse
SE	Evolução da conformidade no tempo (dashboard)	1	12	Baixa	4	GET /audits/summary-over-time
SE	Progresso da auditoria (consulta e stream)	1	6	Baixa	4	GET /audits/{id}/progress|stream
SE	Interpretação de comando em linguagem natural	1	10	Baixa	4	POST /audits/interpret
SE	Experimento de avaliação (precisão/recall)	1	24	Média	5	experiment/run_experiment.py, metrics.py
CE	Listar execuções (filtros e paginação)	1	12	Baixa	3	GET /audits
CE	Consultar configuração do broker	1	4	Baixa	3	GET /config
CE	Consultar sessão/tenant	1	3	Baixa	3	GET /session
CE	Listar tipos de entidade do broker	1	3	Baixa	3	GET /entity-types
CE	Consultar status do catálogo	1	5	Baixa	3	GET /catalog
CE	Listar modelos do catálogo	1	6	Baixa	3	GET /catalog/models
CE	Consultar schema de um modelo	1	4	Baixa	3	GET /catalog/models/{nome}/schema

Tipo	ALI	AIE	EE	SE	CE	Total (PF não ajustados)
Pontos	38	15	22	22	21	118

Fator de ajuste. As 14 características gerais do sistema somam 49 pontos, o que dá VAF = 0,65 + 0,01 × 49 = 1,14 e 134,5 PF ajustados. As notas atribuídas foram: comunicação de dados 5; processamento distribuído 4; processamento complexo (validação, SLM e RAG) 4; múltiplos locais (SaaS e self-hosted) 4; facilidade de instalação 4; entrada de dados online 4; facilidade de mudança 4; e 3 ou menos nas demais.

Premissas e decisões de contagem

O agregado de auditoria (`audit_runs`, `entity_reports`, `violations`, `recommendations`) foi contado como um único ALI com 4 RET e 48 DET (51 colunas menos 3 chaves estrangeiras).

`/health` e `logout` não foram contados, pois nenhum dado de negócio cruza a fronteira. A CLI `run` e `status` repetem funções já contadas na API e não entram em duplicidade.

As telas do frontend são a interface das mesmas funções, não funções novas.

A.3 Modelo COCOMO II e equipe de QA

O tamanho entra no COCOMO II em milhares de linhas de código (KSLOC). Convertendo os 118 PF não ajustados com 24 linhas por ponto de função (valor de referência para Python), chega-se a 2,83 KSLOC. Como conferência, o backend tem 3050 instruções medidas pelo coverage.py, ou seja, a estimativa por APF ficou 7% abaixo do tamanho real do backend, o que valida a contagem.

Equação usada (Post-Architecture): PM = 2,94 × KSLOC^E × ΠEM, com E = 0,91 + 0,01 × ΣSF, e TDEV = 3,67 × PM^(0,28 + 0,2 × (E − 0,91)). Valores dos fatores da calibração COCOMO II.2000.

Fator de escala	Nível	Valor	Justificativa
PREC	Baixo	4,96	SLM/RAG para conformidade FIWARE é pouco precedido na equipe
FLEX	Nominal	3,04	requisitos negociáveis com o orientador
RESL	Nominal	4,24	riscos tratados em ADRs, sem análise formal
TEAM	Alto	2,19	equipe pequena e coesa
PMAT	Baixo	6,24	processo ainda informal (CMM nível 1)

Multiplicador de esforço	Nível	Valor
RELY	Alto	1,10
CPLX	Alto	1,17
PVOL	Baixo	0,87
APEX	Baixo	1,10
PLEX	Baixo	1,09
TOOL	Alto	0,90
SCED	Baixo	1,14
Demais 10 multiplicadores	Nominal	1,00

Resultado	Tamanho pela APF	Sensibilidade: tamanho medido
KSLOC	2,83	3,05
Expoente E	1,1167	1,1167
Multiplicador total (EAF)	1,3774	1,3774
Esforço (pessoas-mês)	12,95	14,07
Prazo (meses)	8,36	8,58
Equipe média	1,55	1,64
Esforço de QA (25%)	3,24 PM · 492 h	3,52 PM · 535 h
Equipe de QA	1,0 testador por 3,3 meses	1,0 testador por 3,4 meses

> **Como ler a estimativa de QA**
> O COCOMO II estima o esforço de **desenvolver** o sistema (13,0 PM). O esforço de teste foi derivado dele com duas premissas do grupo, ambas editáveis no script: 25% do esforço total dedicado a verificação e validação, testes concentrados nos 40% finais do cronograma, e 152 horas por pessoa-mês. Resultado: cerca de **492 horas**, o equivalente a um testador em tempo integral por 3,3 meses. A variação de 7% entre as duas bases de tamanho mostra que a conclusão não depende da conversão linhas por PF.

Limitações. (1) Os fatores de escala e multiplicadores são julgamentos do grupo, não medições, e o COCOMO II tem incerteza típica de dezenas de pontos percentuais para projetos pequenos. (2) O percentual de 25% para teste é uma premissa e não um resultado do modelo. (3) A conversão de 24 linhas por PF vem de tabelas de referência e não foi calibrada com dados da equipe.

Módulo B — Projeto de Testes Funcionais e Usabilidade

B.1 Técnicas aplicadas e rotinas escolhidas

Cada rotina foi tratada como caixa-preta: o teste conhece só a entrada e a saída esperada, definidas pelo requisito e não pela implementação. A caixa-branca (Módulo C) entra depois, para os ramos que a caixa-preta não alcança.

Rotina	Requisito	Técnicas	Casos
`StructuralValidator.validate`	RF-01	Classes de equivalência; valor-limite da faixa numérica	CT-VAL-01 a 23
`ConformanceStatus.derive`	RF-02	Tabela de decisão (3 condições, 6 regras)	CT-STA-01 a 07
`InterpretAuditCommandUseCase.execute`	RF-03	Tabela de decisão (9 regras); valor-limite da confiança	CT-NL-01 a 14
`POST /audits/interpret`	RF-03	Valor-limite do tamanho do texto; classes de entrada	CT-API-01 a 11
`POST /audits`	RF-04	Classes de equivalência	CT-API-12 a 15
`GET /audits` **e** `/summary-over-time`	RF-05	Valor-limite de paginação; classes de filtro	CT-PAG-01 a 16
`PUT /config`	RF-06	Classes e valor-limite; tabela de decisão causa-efeito	CT-CFG-01 a 19
`POST /auth/login`	RF-07	Classes de equivalência	CT-AUT-01 a 07

B.2 Classes de equivalência: StructuralValidator (RF-01)

A entidade é um dicionário plano e o schema vem do Smart Data Model. Para cada regra do validador existe uma classe válida e uma ou mais inválidas; um caso por classe, mais casos que combinam violações.

Variável	Classes válidas	Classes inválidas	Casos
Propriedade obrigatória	presente e não nula	ausente · presente porém nula	VAL-01 · 02 · 03
Tipo do valor	tipo igual ao do schema	texto onde se espera número · booleano onde se espera número · booleano onde se espera inteiro	VAL-01 · 04 · 05 · 21*
Valor enumerado	dentro do conjunto	fora do conjunto	VAL-01 · 06
Formato date-time	ISO 8601 com hora e fuso	texto livre · data sem hora	VAL-08 · 07 · 22*
Propriedade extra	declarada no schema	não declarada (aviso, não erro)	VAL-01 · 09
Obrigatório alternativo (anyOf)	uma das alternativas presente	nenhuma alternativa presente	VAL-11 · 12 · 13
Combinação	—	várias violações na mesma entidade	VAL-10

* Casos que expõem defeitos (ver Módulo C).

Análise de valor-limite: lanternHeight com mínimo 0 e máximo 20

Caso	Valor	Posição	Resultado esperado
CT-VAL-14	−0,001	abaixo do mínimo	value_out_of_range
CT-VAL-15	0	mínimo	sem violação
CT-VAL-16	0,001	logo acima do mínimo	sem violação
CT-VAL-17	10	nominal	sem violação
CT-VAL-18	19,999	logo abaixo do máximo	sem violação

Caso	Valor	Posição	Resultado esperado
CT-VAL-19	20	máximo	sem violação
CT-VAL-20	20,001	acima do máximo	value_out_of_range

B.3 Tabela de decisão: derivação do status (RF-02)

Condições: C1 modelo encontrado no catálogo; C2 existe violação de severidade ERROR; C3 existem violações e todas são WARNING ou INFO. As combinações impossíveis (C2 e C3 verdadeiras juntas) foram eliminadas.

Regra	C1	C2	C3	Status esperado	Caso
R1	N	—	—	no_model_found (violações são ignoradas)	STA-01 · 02
R2	S	N	N	conformant	STA-03
R3	S	N	S	partial (só WARNING)	STA-04
R4	S	N	S	partial (só INFO)	STA-05
R5	S	S	—	non_conformant (só ERROR)	STA-06
R6	S	S	—	non_conformant (ERROR domina, em qualquer ordem)	STA-07

B.4 Tabela de decisão: interpretação de comando (RF-03)

O caso de uso recebe a saída do interpretador (SLM) e decide se mostra um pedido pronto para confirmar, pede esclarecimento ou lista os tipos do broker. O interpretador e o broker são mocks, então as regras podem ser exercitadas sem Ollama nem Orion.

Regra	SLM no ar	Intenção	Tipo e confiança	Broker	Resultado	Caso
R1	N	qualquer	—	—	devolve como veio; não consulta o broker	NL-01
R2	S	out_of_scope	—	—	zera tipo e id; não consulta o broker	NL-02
R3	S	audit	ok	—	permanece audit; não consulta o broker	NL-03
R4	S	audit	sem tipo	S	vira clarify e lista tipos	NL-04
R5	S	audit	confiança < 0,5	S	vira clarify e zera o tipo	NL-05
R6	S	clarify	—	S	mantém a pergunta do SLM e lista tipos	NL-06
R7	S	clarify	—	N	broker_reachable falso; tipos vazios	NL-07
R8	S	discovery	—	S	lista os tipos do broker	NL-08
R9	S	discovery	—	N	broker_reachable falso; tipos vazios	NL-09

Valor-limite do limiar de confiança (0,5): 0,0 e 0,49 resultam em clarify; 0,5, 0,51 e 1,0 permanecem audit (CT-NL-10 a 14). O teste de 0,5 confirma que o limiar é inclusivo, o que um mutante de < para <= quebraria.

B.5 Entradas da API: classes e valores-limite (RF-03 a RF-07)

POST /audits/interpret: command_text com 1 a 500 caracteres

Caso	Tamanho	Posição	Esperado
CT-API-01	0	abaixo do mínimo	422
CT-API-02	1	mínimo	200

Caso	Tamanho	Posição	Esperado
CT-API-03	2	logo acima do mínimo	200
CT-API-04	499	logo abaixo do máximo	200
CT-API-05	500	máximo	200
CT-API-06	501	acima do máximo	422
CT-API-08	3 espaços	só espaços (classe própria)	422 · hoje retorna 200 (D-06)

GET /audits: paginação (contrato proposto: page ≥ 1 e 1 ≤ page_size ≤ 100)

Caso	Entrada	Esperado	Observado hoje
CT-PAG-01	page=1	200	confere
CT-PAG-02 · 03	page=0 · page=−1	422	200, valor repassado ao repositório (D-01)
CT-PAG-04	page=abc	422	confere
CT-PAG-05	page_size=1	200	confere
CT-PAG-06 · 07	page_size=0 · −5	422	200 (D-01)
CT-PAG-08	page_size=100	200	confere
CT-PAG-09 · 10	page_size=101 · 1.000.000	422	200 (D-01)
CT-PAG-11 · 12	status=failed · status=invalido	200 · 422	confere
CT-PAG-16	summary-over-time?entity_type=	422	500 (D-02)

PUT /config: classes e valores-limite dos três campos

Campo	Regra no schema	Casos
`broker_url`	começa com http:// ou https:// e não tem espaço	válidos: CFG-01, 02 · inválidos: ftp (03), sem esquema (04), com espaço (05), só esquema (06)
`fiware_service`	letras, dígitos e _, de 0 a 50 caracteres	AVL: 0 (07), 1 (08), 50 (09), 51 (10) · inválidos: hífen (11), acento (12) · válido com _ (13)
`fiware_service_path`	começa com /, níveis separados por /, sem / final	válidos: / (14), /a/b (15) · inválidos: sem barra (16), barra final (17), // (18)

Causa-efeito de PUT /config. Causas: C1 URL válida, C2 service válido, C3 path válido, C4 chave de criptografia configurada. Efeitos: E1 200 e dado persistido (C1, C2, C3 e C4 verdadeiras); E2 422 e nada persistido (qualquer entre C1, C2, C3 falsa, com C4 verdadeira); E3 503 (C1, C2 e C3 verdadeiras e C4 falsa, caso CT-CFG-19). Os casos de 422 também verificam que `create` nunca é chamado.

B.6 Roteiro e especificação de casos de teste

Os 125 casos estão no Anexo A, no formato da disciplina: ID, nome, descrição (passos), pré-condição, dados, resultado esperado (oráculo), prioridade e status. Dois exemplos:

Campo	CT-VAL-04	CT-API-06
Nome	Tipo errado (string onde se espera number)	501 caracteres: limite superior +
Descrição	Chamar `StructuralValidator().validate(entidade, schema)` e inspecionar as violações	POST /audits/interpret com `command_text` de 501 caracteres
Pré-condição	Schema Streetlight de teste carregado	API no ar com dependências simuladas, sem autenticação

Campo	CT-VAL-04	CT-API-06
Dados de teste	`lanternHeight = "alta"` (schema: number)	`"a" × 501`
Resultado esperado	`type_mismatch` em lanternHeight, severidade ERROR	HTTP 422
Prioridade · Execução	Alta · automatizada (pytest)	Alta · automatizada (pytest)
Status	PASSOU	PASSOU

A coluna “testador” do modelo da aula não foi incluída porque a execução é automatizada; o pytest é o executor e registra o resultado no `junit.xml`.

B.7 Análise de usabilidade e IHC

Método. Inspeção heurística com as 10 heurísticas de Nielsen, mais análise de affordance (percebida × falsa) e de comunicabilidade (o que o designer consegue dizer ao usuário pela interface). Foram inspecionados os templates e componentes Angular das 7 telas: login, configuração, painel, auditorias, assistente, relatório e catálogo. Severidade na escala de Nielsen: 0 não é problema, 1 cosmético, 2 menor, 3 maior, 4 catastrófico.

> **Limite da análise**
> A avaliação foi feita sobre o código das telas, por um avaliador, sem sessões com usuários. Cores, contraste e comportamento visual real não foram medidos. Recomenda-se repetir com 3 a 5 avaliadores e um teste com usuários.

Pontos fortes

- **Estados de tela consistentes.** Painel, auditorias, relatório e catálogo têm esqueleto de carregamento, estado vazio com dica e erro com botão de nova tentativa (heurística 1, visibilidade do estado do sistema).
- **Assistente com controle do usuário.** A tela informa “nada é executado sem a sua confirmação” e mostra “Como o sistema entendeu”, com intenção, tipo e confiança (heurísticas 1 e 3). O campo limita a 500 caracteres, igual à API, e o botão fica desabilitado com campo vazio (heurística 5, prevenção de erros).
- **Rótulos corretos nos formulários de login e configuração** (`label` com `for`, `type="email"` e `type="password"`, `autocomplete`) e teste de conexão automático após salvar a configuração.
- **Status em português nos selos** (Conforme, Parcial, Não conforme, Na fila) com cor e texto juntos, sem depender só da cor.

Problemas encontrados

ID	Tela	Heurística	Evidência	Sev.	Recomendação
U-01	Configuração	H9 recuperação de erros · H5 prevenção	Qualquer falha ao salvar mostra “Não foi possível salvar a configuração. Tente novamente.”, inclusive quando a API rejeita o campo (422). As regras (até 50 caracteres, sem hífen, ServicePath começando com /) não aparecem para o usuário. Tentar de novo não resolve.	3	Validar no cliente com as mesmas regras e mostrar a mensagem de cada campo. (D-09)
U-02	Painel	H2 correspondência com o mundo real	O ranking “Tipos com mais violações” exibe `non_conformant_count + partial_count`, que conta entidades, mas o rótulo diz “violações”. A informação induz a erro. (D-07)	3	Trocar o rótulo para “entidades com problema” ou somar violações de fato.

ID	Tela	Heurística	Evidência	Sev.	Recomendação
U-03	Auditorias	H4 consistência	O filtro de status lista valores crus em inglês (`queued`, `running`…), enquanto a tabela usa selos em português; o status `no_entities_found` nem aparece no filtro. (D-08)	2	Usar os mesmos rótulos dos selos e incluir todos os status.
U-04	Auditorias · Catálogo	H6 reconhecimento em vez de memória	O filtro “Tipo” é texto livre e exige o nome exato (o NGSI diferencia maiúsculas). A lista de tipos do broker já existe na API (`/entity-types`).	2	Trocar por seleção com os tipos disponíveis.
U-05	Auditorias · Catálogo	Acessibilidade · H4	Os campos de filtro só têm `placeholder`, sem `label` ou `aria-label`; leitores de tela não os anunciam, e o placeholder some ao digitar.	2	Adicionar rótulo visível ou `aria-label`.
U-06	Relatório	H2 · H4	Os títulos dos grupos de violação mostram `error`, `warning` e `info` crus, em inglês, enquanto o selo equivalente já traduz (“Erro”, “Aviso”). “Nível 1” e “Nível 2” aparecem sem explicação. (D-10)	2	Reaproveitar o rótulo do selo e acrescentar uma dica sobre os níveis.
U-07	Configuração	H2 · comunicabilidade	“Fiware-Service” e “Fiware-ServicePath” são jargão; só o primeiro tem dica, e o ServicePath (obrigatório) não tem exemplo.	2	Placeholders (`/`, `/filial/sp`) e uma dica por campo.
U-08	Configuração	H5 prevenção de erros · affordance	A URL é um campo de texto comum e o teste de conexão só acontece depois de salvar, ou seja, uma configuração errada já foi gravada.	2	Botão “Testar conexão” antes de salvar; `type="url"`.
U-09	Login	H9 · H3	`required` sem mensagem por campo, sem opção de mostrar a senha e sem caminho de recuperação de senha.	1	Mensagens inline e botão de mostrar senha.
U-10	Catálogo	H2	O campo “Subject (ex.: dataModel.Streetlighting)” usa o jargão do Smart Data Models sem ajuda.	1	Dica curta explicando o que é Subject.

Affordance e comunicabilidade

- **Affordance percebida:** botões de envio com destaque, link “Ver relatório” na tabela, alternador de tipo no painel com estado ativo marcado, e botão “Interpretar” que muda para “Interpretando…” e desabilita durante o processamento.
- **Affordance fraca:** “Ver relatório” é um link de texto simples; a linha da tabela inteira não é clicável, o que dificulta o clique em telas de toque. Não foi encontrada affordance falsa evidente (nenhum elemento com aparência de botão sem ação).
- **Comunicabilidade boa:** o assistente diz o que fez (“Como o sistema entendeu”), o quanto confia (percentual e rótulo) e o que não fez (“nada foi executado”), inclusive quando o Ollama está fora do ar.
- **Comunicabilidade ruim:** a configuração não diz por que recusou o dado (U-01) e o painel diz uma coisa e mostra outra (U-02). O designer não conseguiu transmitir a regra do sistema no momento em que o usuário mais precisa dela.

Módulo C — Automação, Carga e Relatório de Qualidade

C.1 Arquitetura da suíte

A suíte nova fica separada da existente: nada em `tests/unit` foi alterado. Cada módulo declara seus casos como dados (classe `Caso`: ID, requisito, técnica, pré-condição, dados, oráculo, prioridade) e os executa com `@pytest.mark.parametrize`, usando o ID do caso como identificador do teste. O mesmo dado alimenta o Anexo A e a matriz de rastreabilidade, então o roteiro nunca fica desatualizado em relação ao código.

Arquivo	Técnica	Casos	Testes	Resultado
`test_bb_01_structural_validator.py`	CE e AVL	23	23	20 passam · 3 defeitos (D-03 a D-05)
`test_bb_02_conformance_status.py`	Tabela de decisão	7	8	8 passam
`test_bb_03_interpret_command.py`	Tabela de decisão e AVL; mocks	14	14	14 passam
`test_bb_04_api_inputs.py`	CE e AVL na borda HTTP; fakes	57	57	48 passam · 9 defeitos (D-01, D-02, D-06)
`test_wb_01_handlers_lambda.py`	Caixa-branca (caminhos); mocks	13	13	13 passam
`test_wb_02_validador_ramos.py`	Caixa-branca (ramos)	8	8	8 passam
`tests/carga/test_carga_smoke.py`	200 requisições concorrentes	1	1	1 passa
`tests/carga/locustfile.py`	Carga e estresse (Locust)	2	—	2 passam (seção C.4)

- **Parametrização:** 118 dos 123 casos automatizados em pytest são parametrizados (`pytest.mark.parametrize` com `ids` igual ao ID do caso); os outros cinco (CT-CFG-19, CT-WB-09, CT-WB-10, CT-WB-21 e CT-CAR-01) são testes simples. Os marcadores `req` e `tecnica` estão registrados no `pyproject.toml`.
- **Isolamento por mocks e fakes:** `AsyncMock` para interpretador, broker e casos de uso; `dependency_overrides` do FastAPI e `httpx.ASGITransport` para a API em memória; `monkeypatch` para os handlers. Nenhum teste novo usa rede, banco ou Ollama.
- **Defeitos como xfail estrito:** um caso que expõe defeito conhecido passa a ser "falha esperada". A suíte fica verde hoje e, no dia em que o defeito for corrigido, o teste vira XPASS estrito e quebra, obrigando a remover a marca.

Como executar

`pip install -e ".[dev]"` e depois `pytest tests/unit tests/qualidade tests/carga --cov=src/auditor`. Os comandos de carga, APF/COCOMO e rastreabilidade estão em `docs/qualidade/README.md`.

C.2 Resultados da execução

Execução	Passaram	Falhas esperadas (xfail)	Falhas inesperadas	Tempo
Baseline: `tests/unit` antes da entrega	480 (1 pulado)	—	0	17 s
Suíte nova: `tests/qualidade` e `tests/carga`	112	12	0	9 s
Suíte completa: unitária + nova	592 (1 pulado)	12	0	25 s

Os 12 casos "falha esperada" são os defeitos D-01 a D-06 (seção C.7). Os testes de integração (`integration`) e de regressão do SLM (`golden`) do projeto exigem Orion, Postgres e Ollama e não foram executados aqui.

> **Observação sobre o ambiente**
> Cinco testes da suíte existente (`test_context_broker_factory`, `test_catalog_index` e `test_catalog_router`) dependem de extras não instalados por padrão (`pysmartdatamodels` e suas dependências, `jsonref` e outras). Eles falham em um ambiente mínimo e passam com o extra instalado. O baseline de 480 foi medido com os extras.

C.3 Cobertura de código

Cobertura de instruções sobre `src/auditor` (coverage.py): 86,6% antes (2641 de 3050 instruções) e 88,7% depois (2704 de 3050). A suíte existente já era forte; o ganho está em pontos específicos e de alto risco.

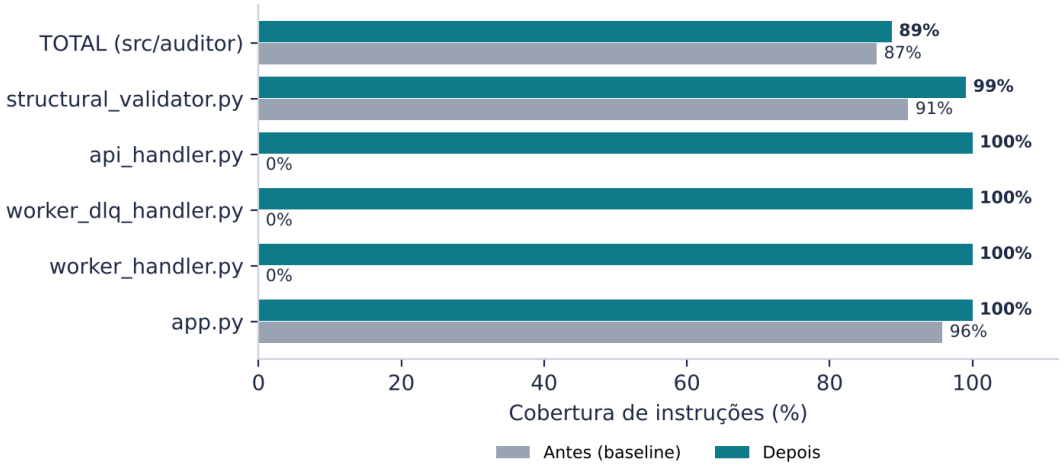


Figura 1 — Cobertura por arquivo, antes e depois (somente arquivos que mudaram)

Arquivo	Instruções	Antes	Depois	Como foi coberto
`structural_validator.py`	111	91%	99%	ramos do anyOf, tipo desconhecido e nomes de tipo (CT-WB-13 a 20)
`api_handler.py`	7	0%	100%	evento do ALB via Mangum (CT-WB-21)
`worker_dlq_handler.py`	23	0%	100%	caminho básico (CT-WB-05 a 08, 10, 12)
`worker_handler.py`	22	0%	100%	caminho básico (CT-WB-01 a 04, 09, 11)
`app.py`	48	96%	100%	criação da app com e sem workers (carga e Lambda)

Cobertura de instruções não prova que os testes são bons; por isso a seção C.5 verifica a eficácia com mutação manual.

C.4 Testes de carga e desempenho

Alvo e escopo. A API real (roteamento, validação e serialização do FastAPI) roda sob o uvicorn com um worker; as dependências externas são fakes em memória, com 500 execuções históricas como massa. O teste mede, portanto, o custo da camada HTTP e de aplicação, não o do Orion, do Postgres nem do SLM. A mistura de tráfego imita a interface: 5 partes de listagem, 3 de painel, 2 de criação de auditoria, 1 de assistente e 1 de health check, com espera de 0,1 a 0,5 s entre ações.

Carga nominal: 100 usuários, 45 s

Endpoint	Requisições	Falhas	Mediana (ms)	p95 (ms)	req/s
GET /audits	4875	0	49	150	110,7

Endpoint	Requisições	Falhas	Mediana (ms)	p95 (ms)	req/s
GET /audits/summary-over-time	2926	0	46	150	66,4
GET /health/live	1016	0	5	19	23,1
POST /audits	1998	0	46	150	45,4
POST /audits/interpret	1010	0	52	150	22,9
Aggregated	11825	0	42	150	268,4

Estresse: 500 usuários, 45 s

Endpoint	Requisições	Falhas	Mediana (ms)	p95 (ms)	req/s
GET /audits	5278	0	1500	1800	117,1
GET /audits/summary-over-time	3135	0	1500	1800	69,6
GET /health/live	1073	0	20	69	23,8
POST /audits	2112	0	1500	1800	46,9
POST /audits/interpret	1034	0	1500	1800	22,9
Aggregated	12632	0	1500	1800	280,3

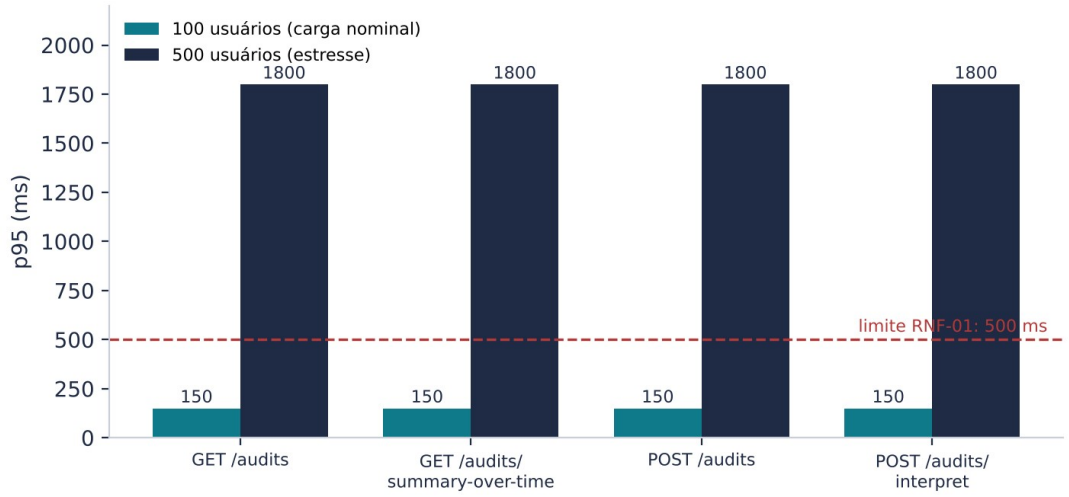


Figura 2 — p95 por endpoint com 100 e 500 usuários simultâneos (linha: limite do RNF-01)

Análise. Com 100 usuários a API atende ≈ 268 req/s, sem nenhuma falha e com p95 de 150 ms, bem abaixo dos 500 ms do RNF-01 (CT-CAR-02 passa). Com 500 usuários não houve falha (CT-CAR-03 passa, pois o critério é robustez), mas a vazão ficou em ≈ 280 req/s, praticamente a mesma de antes. Como a vazão não cresceu, as requisições passaram a esperar na fila e o p95 subiu para 1,8 s. Esse é o comportamento esperado de um serviço saturado: o ponto de saturação desta configuração (1 worker, 2 vCPU compartilhadas com o gerador de carga) fica perto de 280 req/s, e o limite de 500 ms deixa de valer a partir de poucas centenas de usuários simultâneos.

> **Limitações e recomendação**
>
> Gerador e servidor dividiram a mesma máquina, então os números são conservadores. Em produção o Lambda escala por instância, o que muda o perfil; o teste serve para dimensionar a camada de aplicação e como linha de base de regressão. Dois riscos ficaram fora da medição: o tempo do SLM (principal gargalo de `POST /audits/interpret`) e o custo de consultas reais no Postgres. A ausência de limite em `page_size` (D-01) piora o segundo.

C.5 Eficácia da suíte: verificação por mutação manual

Para saber se os testes realmente detectam erros, foram introduzidos seis defeitos artificiais (mutantes) no código-fonte, um de cada vez, e a suíte nova foi executada. Todos foram detectados, e o código original foi restaurado em seguida.

Mutante	Alteração	Detectado por
M1	Limite superior: `value > maximum` → >=	CT-VAL-19 (valor 20, o máximo)
M2	Limite inferior: `value < minimum` → <=	CT-VAL-15 (valor 0, o mínimo)
M3	Limiar de confiança: < → <=	CT-NL-12 (confiança 0,5)
M4	Status: `any(ERROR)` → `all(ERROR)`	CT-STA-07 (ERROR misturado com WARNING e INFO)
M5	Worker deixa de reportar a mensagem que falhou	CT-WB-03 e CT-WB-04
M6	Limite do comando: `max_length` + 1	CT-API-06 (501 caracteres)

Seis mutantes detectados de seis. Os casos de valor-limite foram os que mais pegaram: nenhum teste do meio da faixa distinguiria > de >=.

C.6 Matriz de rastreabilidade

Requisito → caso de teste → script pytest, com o resultado da última execução. A versão completa, com os 125 casos, está em `docs/qualidade/resultados/matriz_rastreabilidade.csv` e é gerada por `scripts/qualidade/matriz_rastreabilidade.py`, que falha se algum requisito ficar sem caso ou se houver ID duplicado.

Req.	Descrição	Casos	Scripts	Resultado
RF-01	Validar estruturalmente uma entidade NGSI-v2 contra o schema do Smart Data Model	CT-VAL-01–23; CT-WB-13–20	test_bb_01, test_wb_02	28 passam · 3 defeitos
RF-02	Derivar o status de conformidade (conformant/partial/non_conformant/no_model_found) das violações	CT-STA-01–07	test_bb_02	7 passam
RF-03	Interpretar comando em linguagem natural sem executá-lo, pedindo esclarecimento quando ambíguo	CT-NL-01–14; CT-API-01–11	test_bb_03, test_bb_04	24 passam · 1 defeitos
RF-04	Iniciar uma auditoria de forma assíncrona (202 + run_id) para um tipo ou para todos	CT-API-12–15	test_bb_04	3 passam · 1 defeitos
RF-05	Listar e consultar execuções com filtros, paginação e evolução no tempo	CT-PAG-01–16	test_bb_04	9 passam · 7 defeitos
RF-06	Configurar o Context Broker e o tenant (modo self-hosted)	CT-CFG-01–19	test_bb_04	19 passam
RF-07	Autenticar o usuário (modo SaaS) com sessão segura	CT-AUT-01–07	test_bb_04	7 passam
RF-08	Processar auditorias em segundo plano (SQS→Lambda) com reentrega parcial e dead-letter	CT-WB-01–12, 21	test_wb_01	13 passam
RNF-01	Desempenho: p95 < 500 ms e 0 % de falhas sob a carga nominal	CT-CAR-01–02	test_carga_smoke, locustfile	2 passam
RNF-02	Robustez: a API não falha (0 % de erros) sob carga de estresse	CT-CAR-03	locustfile	1 passam

C.7 Relatório de defeitos

Severidade: Alta (erro 500, risco à disponibilidade ou resposta errada de negócio), Média (resultado incorreto em caso de borda ou informação enganosa), Baixa (inconsistência sem dano direto). D-01 a D-06 foram reproduzidos por execução e têm caso automatizado; D-07 a D-10 vêm da inspeção do código das telas e não têm teste automatizado.

ID	Defeito	Evidência	Sev.	Caso	Recomendação
D-01	Paginação sem validação em `GET /audits`	`page=0`, `page=-1`, `page_size=0`, `-5` e `1000000` retornam 200 e chegam ao repositório. Pelo código, o repositório calcula `offset = (page−1) × page_size` e `limit = page_size`, o que gera valores negativos (erro de SQL) ou consulta ilimitada. Não foi executado contra Postgres real.	Alta	CT-PAG-02, 03, 06, 07, 09, 10	`Query(1, ge=1)` e `Query(20, ge=1, le=100)`
D-02	`GET /audits/summary-over-time?entity_type=` retorna 500	`EntityType("")` lança `InvalidEntityTypeError`, que não é tratada e vira erro 500 em vez de 422.	Alta	CT-PAG-16	`Query(min_length=1)` ou handler que devolva 422
D-03	Booleano aceito como `integer`	`{"circuitNumber": true}` com `type: integer` não gera violação; para `number` o validador já rejeita booleano. Em Python `bool` é subclasse de `int`.	Média	CT-VAL-21	Excluir `bool` também no tipo `integer`
D-04	Data sem hora aceita como `date-time`	`"2026-10-01"` e `"20260101T000000"` passam. O `date-time` do JSON Schema é RFC 3339 (data, hora e fuso); o validador usa `datetime.fromisoformat`, mais permissivo. Resulta em falso “conforme”.	Média	CT-VAL-22	Validar com expressão RFC 3339
D-05	`NaN` passa na checagem de faixa	`NaN < min` e `NaN > max` são falsos, então `lanternHeight = NaN` não gera `value_out_of_range`.	Baixa	CT-VAL-23	Rejeitar não finitos com `math.isfinite`
D-06	Entradas só com espaços são aceitas	`command_text = "  "` e `entity_type = "  "` retornam 200/202 e seguem para o SLM ou para a fila. A interface já bloqueia o primeiro caso, mas a API não.	Baixa	CT-API-08, CT-API-15	`str_strip_whitespace` ou validador no schema
D-07	Painel rotula entidades como “violações”	`non_conformant_count + partial_count` conta entidades, e o texto diz “violações”.	Média	— (inspeção)	Corrigir o rótulo ou a conta
D-08	Filtro de status em inglês e incompleto	Mostra `queued`, `running`… e omite `no_entities_found`.	Baixa	— (inspeção)	Usar rótulos dos selos; incluir todos os status
D-09	Configuração esconde a causa do erro	Todo erro de salvamento vira “Tente novamente”, inclusive o 422 por campo inválido.	Média	— (inspeção; regras em CT-CFG-10 e 11)	Validação no cliente e mensagem por campo
D-10	Títulos de severidade crus no relatório	`error`, `warning` e `info` aparecem em inglês.	Baixa	— (inspeção)	Reaproveitar o rótulo do selo

> **Observação O-01 (capacidade, não é defeito)**
> Com 1 worker e 2 vCPU a API satura em ≈ 280 req/s e, acima disso, a latência cresce de forma linear sem gerar erros (seção C.4). Recomenda-se dimensionar a concorrência reservada do Lambda e repetir o teste no ambiente de homologação.

Situação: os defeitos permanecem abertos. Esta entrega documenta e prende cada um com teste; as correções, em sua maioria de uma linha, ficam para um commit separado, o que evita misturar mudança de comportamento da API com a entrega de qualidade.

Conclusão

Aplicar as técnicas da disciplina ao próprio TCC mostrou que a suíte existente era boa onde olhava (domínio e roteadores, 87% de cobertura), mas deixava três pontos de risco sem proteção: as funções Lambda de produção, as bordas de entrada da API e o comportamento sob carga. A nova suíte fecha esses pontos, expõe seis defeitos reproduzíveis e quatro problemas de interface, e deixa cada um preso a um caso de teste rastreável a um requisito.

Atendimento ao enunciado

Exigência do enunciado	Onde está
Plano de Testes (ISO/IEC 29119): escopo, itens testados e não testados, estratégia, riscos, ambiente, critérios	Seção 1 e A.1
Estimativa de esforço por APF e COCOMO II (equipe de QA e horas)	A.2 e A.3 · `scripts/qualidade/apf_cocomo.py`
Classes de equivalência, valor-limite e tabela de decisão em pelo menos 2 rotinas	B.2 a B.5 (4 rotinas e 4 endpoints)
Análise de usabilidade e IHC: affordance, comunicabilidade e prevenção de erros	B.7
Roteiro de casos de teste formais (ID, pré-condição, passos, dados, oráculo)	B.6 e Anexo A (125 casos)
Pytest com `parametrize`, testes unitários e caixa-branca, isolamento por mocks	C.1 · `tests/qualidade/`
Testes de carga e desempenho com tempo de resposta e taxa de erro	C.4 · `tests/carga/`
Matriz de rastreabilidade e relatório de defeitos e cobertura	C.3, C.6 e C.7

Próximos passos

- Corrigir D-01 e D-02 (um `Query(...)` cada) e remover a marca `xfail` dos casos correspondentes; em seguida D-03 a D-06.
- Colocar `pytest tests/unit tests/qualidade tests/carga` no pipeline e definir piso de cobertura de 85%.
- Repetir a carga no ambiente de homologação, com Postgres e Lambda reais, e medir o tempo do SLM separadamente.
- Corrigir os problemas de interface D-07 a D-10 e repetir a avaliação de usabilidade com 3 a 5 avaliadores e um teste com usuários.

Referências

- ISO/IEC/IEEE 29119-3:2021. Software and systems engineering — Software testing — Part 3: Test documentation.
- IFPUG. Function Point Counting Practices Manual (CPM). International Function Point Users Group.
- BOEHM, B. W. et al. Software Cost Estimation with COCOMO II. Prentice Hall, 2000.
- MYERS, G. J. The Art of Software Testing. Wiley, 1979.
- NIELSEN, J. Usability Inspection Methods. Wiley, 1994; 10 Usability Heuristics for User Interface Design, Nielsen Norman Group.
- DE SOUZA, C. S. The Semiotic Engineering of Human-Computer Interaction. MIT Press, 2005 (comunicabilidade).
- NORMAN, D. A. The Design of Everyday Things. Basic Books, 2013 (affordance).
- NORIEGA, C. Aula 03 — Teste Funcional ou Caixa-Preta (slides da disciplina).
- IETF. RFC 3339: Date and Time on the Internet: Timestamps, 2002.
- Documentação do pytest, pytest-cov e Locust (docs.pytest.org; docs.locust.io).

Anexo A — Roteiro completo dos casos de teste

Formato da disciplina (ID, nome, descrição, pré-condição, dados, resultado esperado, prioridade, status). Req. = requisito rastreado; Téc. = técnica (CE classes de equivalência, AVL valor-limite, TD tabela de decisão, CB caixa-branca). Execução automatizada; o resultado vem do `junit.xml` e do Locust.

ID	Req. · Téc.	Nome	Descrição (passos)	Pré-condição	Dados de teste	Resultado esperado	Prior.	Status
CT-VAL-01	RF-01 CE	Entidade totalmente valida	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	id,type,status=ok	Nenhuma violação	Alta	PASSOU
CT-VAL-02	RF-01 CE	Obrigatório ausente	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	sem 'status'	required_property_missing em status (ERROR)	Alta	PASSOU
CT-VAL-03	RF-01 CE	Obrigatório presente porém nulo	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	status=None	required_property_missing em status (ERROR)	Média	PASSOU
CT-VAL-04	RF-01 CE	Tipo errado (string onde se espera number)	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	lanternHeight='alta'	type_mismatch em lanternHeight (ERROR)	Alta	PASSOU
CT-VAL-05	RF-01 CE	Booleano onde se espera number	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	lanternHeight=true	type_mismatch em lanternHeight (ERROR)	Média	PASSOU
CT-VAL-06	RF-01 CE	Valor fora do enum	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	status='quebrado'	enum_violation em status (ERROR)	Alta	PASSOU
CT-VAL-07	RF-01 CE	date-time inválido (texto livre)	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	dateLastLampChange='ontem'	format_invalid (ERROR)	Média	PASSOU
CT-VAL-08	RF-01 CE	date-time válido com offset	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	dateLastLampChange='2026-10-01T10:00:00Z'	Nenhuma violação	Média	PASSOU
CT-VAL-09	RF-01 CE	Propriedade fora do schema	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	campoExtra='x'	unexpected_property (WARNING)	Média	PASSOU

ID	Req. · Téc.	Nome	Descrição (passos)	Pré-condição	Dados de teste	Resultado esperado	Prior.	Status
CT-VAL-10	RF-01 CE	Várias violações simultâneas	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	sem status + altura em texto + extra	3 violações: missing, type_mismatch, unexpected_property	Alta	PASSOU
CT-VAL-11	RF-01 CE	Obrigatório alternativo (anyOf) satisfeito por location	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	location presente	Nenhuma violação	Média	PASSOU
CT-VAL-12	RF-01 CE	Obrigatório alternativo (anyOf) satisfeito por address	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	address presente	Nenhuma violação	Média	PASSOU
CT-VAL-13	RF-01 CE	Nenhuma alternativa do anyOf satisfeita	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	sem location e sem address	Reporta só a 1ª alternativa: location ausente (ERROR)	Média	PASSOU
CT-VAL-14	RF-01 AVL	lanternHeight no limite inferior − (abaixo de 0)	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	lanternHeight=-0.001	value_out_of_range (ERROR)	Alta	PASSOU
CT-VAL-15	RF-01 AVL	lanternHeight no limite inferior (0)	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	lanternHeight=0	Nenhuma violação	Alta	PASSOU
CT-VAL-16	RF-01 AVL	lanternHeight no limite inferior + (logo acima de 0)	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	lanternHeight=0.001	Nenhuma violação	Alta	PASSOU
CT-VAL-17	RF-01 AVL	lanternHeight no valor nominal (10)	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	lanternHeight=10	Nenhuma violação	Alta	PASSOU
CT-VAL-18	RF-01 AVL	lanternHeight no limite superior − (logo abaixo de 20)	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	lanternHeight=19.999	Nenhuma violação	Alta	PASSOU
CT-VAL-19	RF-01 AVL	lanternHeight no limite superior (20)	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	lanternHeight=20	Nenhuma violação	Alta	PASSOU
CT-VAL-20	RF-01 AVL	lanternHeight no limite superior + (acima de 20)	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	lanternHeight=20.001	value_out_of_range (ERROR)	Alta	PASSOU

ID	Req. · Téc.	Nome	Descrição (passos)	Pré-condição	Dados de teste	Resultado esperado	Prior.	Status
CT-VAL-21	RF-01 CE	Booleano aceito como integer	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	circuitNumber=true (schema integer)	type_mismatch (para number o booleano já é rejeitado)	Média	DEFEITO CONHECIDO D-03
CT-VAL-22	RF-01 CE	Data sem hora aceita como date-time	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	dateLastLampChange='2026-10-01'	format_invalid (RFC 3339 exige data e hora)	Média	DEFEITO CONHECIDO D-04
CT-VAL-23	RF-01 AVL	NaN passa na checagem de faixa	Chamar StructuralValidator().validate(entidade, schema) e inspecionar as violacoes.	Schema Streetlight de teste carregado; validador instanciado.	lanternHeight=NaN	value_out_of_range (NaN não é um número válido na faixa)	Baixa	DEFEITO CONHECIDO D-05
CT-STA-01	RF-02 TD	R1: modelo inexistente, sem violações	Chamar ConformanceStatus.derive(violacoes, model_found=...).	Nenhuma (função pura de domínio).	model_found=N; violações=[]	no_model_found	Alta	PASSOU
CT-STA-02	RF-02 TD	R1: modelo inexistente ignora violações	Chamar ConformanceStatus.derive(violacoes, model_found=...).	Nenhuma (função pura de domínio).	model_found=N; violações=[ERROR]	no_model_found	Alta	PASSOU
CT-STA-03	RF-02 TD	R2: modelo existe, sem violações	Chamar ConformanceStatus.derive(violacoes, model_found=...).	Nenhuma (função pura de domínio).	model_found=S; violações=[]	conformant	Alta	PASSOU
CT-STA-04	RF-02 TD	R3: apenas WARNING	Chamar ConformanceStatus.derive(violacoes, model_found=...).	Nenhuma (função pura de domínio).	model_found=S; violações=[WARNING]	partial	Média	PASSOU
CT-STA-05	RF-02 TD	R4: apenas INFO	Chamar ConformanceStatus.derive(violacoes, model_found=...).	Nenhuma (função pura de domínio).	model_found=S; violações=[INFO]	partial	Média	PASSOU
CT-STA-06	RF-02 TD	R5: apenas ERROR	Chamar ConformanceStatus.derive(violacoes, model_found=...).	Nenhuma (função pura de domínio).	model_found=S; violações=[ERROR]	non_conformant	Alta	PASSOU
CT-STA-07	RF-02 TD	R6: ERROR junto com WARNING e INFO	Chamar ConformanceStatus.derive(violacoes, model_found=...).	Nenhuma (função pura de domínio).	model_found=S; violações=[WARNING, ERROR, INFO]	non_conformant (ERROR domina, independe da ordem)	Alta	PASSOU
CT-NL-01	RF-03 TD	R1: SLM indisponível	Executar o caso de uso com o comando 'audite Streetlight' e a saída simulada do interpretador.	Interpretador e broker substituídos por mocks (AsyncMock).	interpreter_available=N	Devolve a interpretação sem tocar no broker	Alta	PASSOU
CT-NL-02	RF-03 TD	R2: pedido fora do escopo	Executar o caso de uso com o comando 'audite Streetlight' e a saída simulada do interpretador.	Interpretador e broker substituídos por mocks (AsyncMock).	intent=out_of_scope; tipo=Streetlight	Zera tipo/id e não consulta o broker	Alta	PASSOU
CT-NL-03	RF-03 TD	R3: auditoria clara e confiante	Executar o caso de uso com o comando 'audite Streetlight' e a saída simulada do interpretador.	Interpretador e broker substituídos por mocks (AsyncMock).	intent=audit; tipo=Streetlight; conf=0.9	Permanece audit com o tipo; não consulta o broker	Alta	PASSOU

ID	Req. · Téc.	Nome	Descrição (passos)	Pré-condição	Dados de teste	Resultado esperado	Prior.	Status
CT-NL-04	RF-03 TD	R4: auditoria sem tipo	Executar o caso de uso com o comando 'audite Streetlight' e a saída simulada do interpretador.	Interpretador e broker substituídos por mocks (AsyncMock).	intent=audit; tipo=None	Vira clarify e lista tipos do broker	Alta	PASSOU
CT-NL-05	RF-03 TD	R5: auditoria com confiança baixa	Executar o caso de uso com o comando 'audite Streetlight' e a saída simulada do interpretador.	Interpretador e broker substituídos por mocks (AsyncMock).	intent=audit; conf=0.1	Vira clarify e zera o tipo	Alta	PASSOU
CT-NL-06	RF-03 TD	R6: clarify mantém a pergunta do SLM	Executar o caso de uso com o comando 'audite Streetlight' e a saída simulada do interpretador.	Interpretador e broker substituídos por mocks (AsyncMock).	intent=clarify; pergunta='Qual tipo?'	Pergunta preservada + tipos do broker	Média	PASSOU
CT-NL-07	RF-03 TD	R7: clarify com broker fora do ar	Executar o caso de uso com o comando 'audite Streetlight' e a saída simulada do interpretador.	Interpretador e broker substituídos por mocks (AsyncMock).	intent=clarify; broker lança ContextBrokerUnavailableError	broker_reachable=False e tipos vazios	Média	PASSOU
CT-NL-08	RF-03 TD	R8: discovery com broker no ar	Executar o caso de uso com o comando 'audite Streetlight' e a saída simulada do interpretador.	Interpretador e broker substituídos por mocks (AsyncMock).	intent=discovery	Lista os tipos do broker	Alta	PASSOU
CT-NL-09	RF-03 TD	R9: discovery com broker fora do ar	Executar o caso de uso com o comando 'audite Streetlight' e a saída simulada do interpretador.	Interpretador e broker substituídos por mocks (AsyncMock).	intent=discovery; broker indisponível	broker_reachable=False e tipos vazios	Média	PASSOU
CT-NL-10	RF-03 AVL	Confiança 0.0 (limiar 0.5)	Executar o caso de uso com o comando 'audite Streetlight' e a saída simulada do interpretador.	Interpretador e broker substituídos por mocks (AsyncMock).	intent=audit; tipo=Streetlight; confidence=0.0	intent=clarify	Alta	PASSOU
CT-NL-11	RF-03 AVL	Confiança 0.49 (limiar 0.5)	Executar o caso de uso com o comando 'audite Streetlight' e a saída simulada do interpretador.	Interpretador e broker substituídos por mocks (AsyncMock).	intent=audit; tipo=Streetlight; confidence=0.49	intent=clarify	Alta	PASSOU
CT-NL-12	RF-03 AVL	Confiança 0.5 (limiar 0.5)	Executar o caso de uso com o comando 'audite Streetlight' e a saída simulada do interpretador.	Interpretador e broker substituídos por mocks (AsyncMock).	intent=audit; tipo=Streetlight; confidence=0.5	intent=audit	Alta	PASSOU
CT-NL-13	RF-03 AVL	Confiança 0.51 (limiar 0.5)	Executar o caso de uso com o comando 'audite Streetlight' e a saída simulada do interpretador.	Interpretador e broker substituídos por mocks (AsyncMock).	intent=audit; tipo=Streetlight; confidence=0.51	intent=audit	Alta	PASSOU
CT-NL-14	RF-03 AVL	Confiança 1.0 (limiar 0.5)	Executar o caso de uso com o comando 'audite Streetlight' e a saída simulada do interpretador.	Interpretador e broker substituídos por mocks (AsyncMock).	intent=audit; tipo=Streetlight; confidence=1.0	intent=audit	Alta	PASSOU

Anexo A — Roteiro de casos de teste

ID	Req. · Téc.	Nome	Descrição (passos)	Pré-condição	Dados de teste	Resultado esperado	Prior.	Status
CT-API-01	RF-03 AVL	Vazio (0 caracteres): limite inferior –	POST /audits/interpret com {command_text: <texto>}.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	command_text=''… (len=0)	HTTP 422	Alta	PASSOU
CT-API-02	RF-03 AVL	1 caractere: limite inferior	POST /audits/interpret com {command_text: <texto>}.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	command_text='a'… (len=1)	HTTP 200	Alta	PASSOU
CT-API-03	RF-03 AVL	2 caracteres: limite inferior +	POST /audits/interpret com {command_text: <texto>}.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	command_text='ab'… (len=2)	HTTP 200	Alta	PASSOU
CT-API-04	RF-03 AVL	499 caracteres: limite superior –	POST /audits/interpret com {command_text: <texto>}.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	command_text='aaaaaaaaaaa'… (len=499)	HTTP 200	Alta	PASSOU
CT-API-05	RF-03 AVL	500 caracteres: limite superior	POST /audits/interpret com {command_text: <texto>}.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	command_text='aaaaaaaaaaa'… (len=500)	HTTP 200	Alta	PASSOU
CT-API-06	RF-03 AVL	501 caracteres: limite superior +	POST /audits/interpret com {command_text: <texto>}.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	command_text='aaaaaaaaaaa'… (len=501)	HTTP 422	Alta	PASSOU
CT-API-07	RF-03 CE	Frase típica em português	POST /audits/interpret com {command_text: <texto>}.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	command_text='audite os po'… (len=23)	HTTP 200	Média	PASSOU
CT-API-08	RF-03 CE	Somente espaços (comando semanticamente vazio)	POST /audits/interpret com {command_text: <texto>}.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	command_text=' '… (len=3)	HTTP 422	Média	DEFEITO CONHECIDO D-06
CT-API-09	RF-03 CE	Campo command_text ausente	POST /audits/interpret com {command_text: <texto>}.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	{}	HTTP 422	Média	PASSOU
CT-API-10	RF-03 CE	command_text com tipo errado (número)	POST /audits/interpret com {command_text: <texto>}.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	{'command_text': 123}	HTTP 422	Média	PASSOU
CT-API-11	RF-03 CE	command_text nulo	POST /audits/interpret com {command_text: <texto>}.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	{'command_text': None}	HTTP 422	Média	PASSOU

ID	Req. · Téc.	Nome	Descrição (passos)	Pré-condição	Dados de teste	Resultado esperado	Prior.	Status
CT-API-12	RF-04 CE	Auditoria de um tipo	POST /audits com o corpo indicado.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	{'entity_type': 'Streetlight'}	HTTP 202	Alta	PASSOU
CT-API-13	RF-04 CE	Auditoria ampla (corpo vazio)	POST /audits com o corpo indicado.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	{}	HTTP 202	Alta	PASSOU
CT-API-14	RF-04 CE	entity_type com tipo errado (número)	POST /audits com o corpo indicado.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	{'entity_type': 5}	HTTP 422	Alta	PASSOU
CT-API-15	RF-04 CE	entity_type só com espaços	POST /audits com o corpo indicado.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	{'entity_type': ' '}	HTTP 422	Alta	DEFEITO CONHECIDO D-06
CT-PAG-01	RF-05 AVL	page=1 (limite inferior)	GET /audits com a query indicada.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	GET /audits?page=1	HTTP 200	Média	PASSOU
CT-PAG-02	RF-05 AVL	page=0 (abaixo do limite)	GET /audits com a query indicada.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	GET /audits?page=0	HTTP 422	Alta	DEFEITO CONHECIDO D-01
CT-PAG-03	RF-05 AVL	page=-1	GET /audits com a query indicada.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	GET /audits?page=-1	HTTP 422	Alta	DEFEITO CONHECIDO D-01
CT-PAG-04	RF-05 CE	page=abc (tipo inválido)	GET /audits com a query indicada.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	GET /audits?page=abc	HTTP 422	Média	PASSOU
CT-PAG-05	RF-05 AVL	page_size=1 (limite inferior)	GET /audits com a query indicada.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	GET /audits?page_size=1	HTTP 200	Média	PASSOU
CT-PAG-06	RF-05 AVL	page_size=0 (abaixo do limite)	GET /audits com a query indicada.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	GET /audits?page_size=0	HTTP 422	Alta	DEFEITO CONHECIDO D-01
CT-PAG-07	RF-05 AVL	page_size=-5	GET /audits com a query indicada.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	GET /audits?page_size=-5	HTTP 422	Alta	DEFEITO CONHECIDO D-01

ID	Req. · Téc.	Nome	Descrição (passos)	Pré-condição	Dados de teste	Resultado esperado	Prior.	Status
CT-PAG-08	RF-05 AVL	page_size=100 (limite superior)	GET /audits com a query indicada.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	GET /audits?page_size=100	HTTP 200	Média	PASSOU
CT-PAG-09	RF-05 AVL	page_size=101 (acima do limite)	GET /audits com a query indicada.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	GET /audits?page_size=101	HTTP 422	Alta	DEFEITO CONHECIDO D-01
CT-PAG-10	RF-05 AVL	page_size=1000000 (carga abusiva)	GET /audits com a query indicada.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	GET /audits?page_size=1000000	HTTP 422	Alta	DEFEITO CONHECIDO D-01
CT-PAG-11	RF-05 CE	Filtro status válido	GET /audits com a query indicada.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	GET /audits?status=failed	HTTP 200	Média	PASSOU
CT-PAG-12	RF-05 CE	Filtro status inexistente	GET /audits com a query indicada.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	GET /audits?status=invalido	HTTP 422	Média	PASSOU
CT-PAG-13	RF-05 CE	Sem parâmetros (padrões)	GET /audits com a query indicada.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	GET /audits?	HTTP 200	Média	PASSOU
CT-PAG-14	RF-05 CE	summary-over-time com tipo válido	GET com a query indicada.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	GET /audits/summary-over-time?entity_type=Streetlight	HTTP 200	Alta	PASSOU
CT-PAG-15	RF-05 CE	summary-over-time sem o parâmetro	GET com a query indicada.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	GET /audits/summary-over-time?	HTTP 422	Alta	PASSOU
CT-PAG-16	RF-05 CE	summary-over-time com tipo vazio	GET com a query indicada.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	GET /audits/summary-over-time?entity_type=	HTTP 422	Alta	DEFEITO CONHECIDO D-02
CT-CFG-01	RF-06 CE	Configuração válida	PUT /config com o corpo válido, trocando apenas o campo indicado.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	broker_url='http://orion:1026'	HTTP 200	Alta	PASSOU
CT-CFG-02	RF-06 CE	broker_url https com porta	PUT /config com o corpo válido, trocando apenas o campo indicado.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	broker_url='https://orion.exempl'	HTTP 200	Média	PASSOU

ID	Req. · Téc.	Nome	Descrição (passos)	Pré-condição	Dados de teste	Resultado esperado	Prior.	Status
CT-CFG-03	RF-06 CE	broker_url com esquema ftp	PUT /config com o corpo válido, trocando apenas o campo indicado.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	broker_url='ftp://orion'	HTTP 422	Média	PASSOU
CT-CFG-04	RF-06 CE	broker_url sem esquema	PUT /config com o corpo válido, trocando apenas o campo indicado.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	broker_url='orion:1026'	HTTP 422	Média	PASSOU
CT-CFG-05	RF-06 CE	broker_url com espaço	PUT /config com o corpo válido, trocando apenas o campo indicado.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	broker_url='http://or ion'	HTTP 422	Média	PASSOU
CT-CFG-06	RF-06 CE	broker_url só com esquema	PUT /config com o corpo válido, trocando apenas o campo indicado.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	broker_url='http://'	HTTP 422	Média	PASSOU
CT-CFG-07	RF-06 AVL	fiware_service vazio (0): limite inferior	PUT /config com o corpo válido, trocando apenas o campo indicado.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	fiware_service=''	HTTP 200	Média	PASSOU
CT-CFG-08	RF-06 AVL	fiware_service com 1 caractere	PUT /config com o corpo válido, trocando apenas o campo indicado.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	fiware_service='a'	HTTP 200	Média	PASSOU
CT-CFG-09	RF-06 AVL	fiware_service com 50 caracteres: limite superior	PUT /config com o corpo válido, trocando apenas o campo indicado.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	fiware_service='aaaaaaaaaaaaaaaaaaaaaaaaaa'	HTTP 200	Média	PASSOU
CT-CFG-10	RF-06 AVL	fiware_service com 51 caracteres: limite superior +	PUT /config com o corpo válido, trocando apenas o campo indicado.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	fiware_service='aaaaaaaaaaaaaaaaaaaaaaaaaaa'	HTTP 422	Média	PASSOU
CT-CFG-11	RF-06 CE	fiware_service com hífen	PUT /config com o corpo válido, trocando apenas o campo indicado.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	fiware_service='open-iot'	HTTP 422	Média	PASSOU
CT-CFG-12	RF-06 CE	fiware_service com acento	PUT /config com o corpo válido, trocando apenas o campo indicado.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	fiware_service='ação'	HTTP 422	Média	PASSOU
CT-CFG-13	RF-06 CE	fiware_service com underscore e dígitos	PUT /config com o corpo válido, trocando apenas o campo indicado.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	fiware_service='tenant_01'	HTTP 200	Média	PASSOU

ID	Req. · Téc.	Nome	Descrição (passos)	Pré-condição	Dados de teste	Resultado esperado	Prior.	Status
CT-CFG-14	RF-06 CE	service_path raiz '/'	PUT /config com o corpo válido, trocando apenas o campo indicado.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	fiware_service_path='/'	HTTP 200	Média	PASSOU
CT-CFG-15	RF-06 CE	service_path com 2 níveis	PUT /config com o corpo válido, trocando apenas o campo indicado.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	fiware_service_path='/a/b'	HTTP 200	Média	PASSOU
CT-CFG-16	RF-06 CE	service_path sem barra inicial	PUT /config com o corpo válido, trocando apenas o campo indicado.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	fiware_service_path='a'	HTTP 422	Média	PASSOU
CT-CFG-17	RF-06 CE	service_path terminando em '/'	PUT /config com o corpo válido, trocando apenas o campo indicado.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	fiware_service_path='/a/'	HTTP 422	Média	PASSOU
CT-CFG-18	RF-06 CE	service_path com '//'	PUT /config com o corpo válido, trocando apenas o campo indicado.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	fiware_service_path='//'	HTTP 422	Média	PASSOU
CT-CFG-19	RF-06 CE	Sem chave de criptografia configurada	Enviar a requisição HTTP e verificar o status.	API no ar com dependências simuladas; sem autenticação (modo self-hosted).	accounts=None (sem BROKER_API_KEY_ENCRYPTION_KEYS)	HTTP 503	Alta	PASSOU
CT-AUT-01	RF-07 CE	Credenciais corretas	POST /auth/login com o corpo indicado.	Modo SaaS; conta user@example.com cadastrada com senha 'senha-correta-123'.	{'login_email': 'user@example.com', 'password': '***'}	HTTP 200	Alta	PASSOU
CT-AUT-02	RF-07 CE	E-mail correto, senha errada	POST /auth/login com o corpo indicado.	Modo SaaS; conta user@example.com cadastrada com senha 'senha-correta-123'.	{'login_email': 'user@example.com', 'password': '***'}	HTTP 401	Alta	PASSOU
CT-AUT-03	RF-07 CE	E-mail inexistente	POST /auth/login com o corpo indicado.	Modo SaaS; conta user@example.com cadastrada com senha 'senha-correta-123'.	{'login_email': 'nao@existe.com', 'password': '***'}	HTTP 401	Alta	PASSOU
CT-AUT-04	RF-07 AVL	Senha vazia	POST /auth/login com o corpo indicado.	Modo SaaS; conta user@example.com cadastrada com senha 'senha-correta-123'.	{'login_email': 'user@example.com', 'password': '***'}	HTTP 401	Alta	PASSOU
CT-AUT-05	RF-07 CE	E-mail em maiúsculas (insensível a caixa)	POST /auth/login com o corpo indicado.	Modo SaaS; conta user@example.com cadastrada com senha 'senha-correta-123'.	{'login_email': 'USER@Example.com', 'password': '***'}	HTTP 200	Alta	PASSOU

ID	Req. · Téc.	Nome	Descrição (passos)	Pré-condição	Dados de teste	Resultado esperado	Prior.	Status
CT-AUT-06	RF-07 CE	Campo password ausente	POST /auth/login com o corpo indicado.	Modo SaaS; conta user@example.com cadastrada com senha 'senha-correta-123'.	{'login_email': 'user@example.com'}	HTTP 422	Alta	PASSOU
CT-AUT-07	RF-07 CE	Corpo vazio	POST /auth/login com o corpo indicado.	Modo SaaS; conta user@example.com cadastrada com senha 'senha-correta-123'.	{}	HTTP 422	Alta	PASSOU
CT-WB-01	RF-08 CB	worker_handler: caminho C1 — Evento sem Records	Invocar _handle_async(worker_handler) com o evento SQS simulado.	Use case substituído por AsyncMock; bootstrap AWS sem variáveis (no-op).	0 registro(s); efeitos=[]	batchItemFailures = []; use case chamado 0x	Alta	PASSOU
CT-WB-02	RF-08 CB	worker_handler: caminho C2 — Um registro processado com êxito	Invocar _handle_async(worker_handler) com o evento SQS simulado.	Use case substituído por AsyncMock; bootstrap AWS sem variáveis (no-op).	1 registro(s); efeitos=['ok']	batchItemFailures = []; use case chamado 1x	Alta	PASSOU
CT-WB-03	RF-08 CB	worker_handler: caminho C3 — Um registro cujo processamento falha	Invocar _handle_async(worker_handler) com o evento SQS simulado.	Use case substituído por AsyncMock; bootstrap AWS sem variáveis (no-op).	1 registro(s); efeitos=['erro']	batchItemFailures = ['m0']; use case chamado 1x	Alta	PASSOU
CT-WB-04	RF-08 CB	worker_handler: caminho C4 — Lote misto (ok, falha, ok): laço continua após a falha	Invocar _handle_async(worker_handler) com o evento SQS simulado.	Use case substituído por AsyncMock; bootstrap AWS sem variáveis (no-op).	3 registro(s); efeitos=['ok', 'erro', 'ok']	batchItemFailures = ['m1']; use case chamado 3x	Alta	PASSOU
CT-WB-05	RF-08 CB	worker_dlq_handler: caminho C1 — Evento sem Records	Invocar _handle_async(worker_dlq_handler) com o evento SQS simulado.	Use case substituído por AsyncMock; bootstrap AWS sem variáveis (no-op).	0 registro(s); efeitos=[]	batchItemFailures = []; use case chamado 0x	Alta	PASSOU
CT-WB-06	RF-08 CB	worker_dlq_handler: caminho C2 — Um registro processado com êxito	Invocar _handle_async(worker_dlq_handler) com o evento SQS simulado.	Use case substituído por AsyncMock; bootstrap AWS sem variáveis (no-op).	1 registro(s); efeitos=['ok']	batchItemFailures = []; use case chamado 1x	Alta	PASSOU
CT-WB-07	RF-08 CB	worker_dlq_handler: caminho C3 — Um registro cujo processamento falha	Invocar _handle_async(worker_dlq_handler) com o evento SQS simulado.	Use case substituído por AsyncMock; bootstrap AWS sem variáveis (no-op).	1 registro(s); efeitos=['erro']	batchItemFailures = ['m0']; use case chamado 1x	Alta	PASSOU
CT-WB-08	RF-08 CB	worker_dlq_handler: caminho C4 — Lote misto (ok, falha, ok): laço continua após a falha	Invocar _handle_async(worker_dlq_handler) com o evento SQS simulado.	Use case substituído por AsyncMock; bootstrap AWS sem variáveis (no-op).	3 registro(s); efeitos=['ok', 'erro', 'ok']	batchItemFailures = ['m1']; use case chamado 3x	Alta	PASSOU
CT-WB-21	RF-08 CB	API Lambda: evento do ALB chega à app FastAPI (Mangum)	Chamar api_handler.handler(evento_ALB, None) com GET /health/live.	Sem variáveis AWS (bootstrap no-op)	evento ALB GET /health/live	statusCode=200 e corpo {"status":"ok"}	Alta	PASSOU

ID	Req. · Téc.	Nome	Descrição (passos)	Pré-condição	Dados de teste	Resultado esperado	Prior.	Status
CT-WB-09	RF-08 CB	Worker repassa run_id/tenant/tipo/id ao use case	Invocar _handle_async com 1 registro e verificar os argumentos.	Use case substituído por AsyncMock; bootstrap AWS sem variáveis (no-op).	registro com entity_type=Streetlight	process(run_id, Tenant, EntityType, None)	Alta	PASSOU
CT-WB-10	RF-08 CB	DLQ marca o run como FAILED com o motivo padrão	Invocar _handle_async da DLQ com 1 registro.	Use case substituído por AsyncMock; bootstrap AWS sem variáveis (no-op).	registro da DLQ	mark_failed(run_id, tenant, reason=MAX_RETRIES_EXCEEDED_REASON)	Alta	PASSOU
CT-WB-11	RF-08 CB	Entrada síncrona do worker (handler)	Chamar handler(event, None) — usa asyncio.run.	Use case substituído por AsyncMock; bootstrap AWS sem variáveis (no-op).	1 registro	{'batchItemFailures': []}	Média	PASSOU
CT-WB-12	RF-08 CB	Entrada síncrona da DLQ (handler)	Chamar handler(event, None) — usa asyncio.run.	Use case substituído por AsyncMock; bootstrap AWS sem variáveis (no-op).	1 registro	{'batchItemFailures': []}	Média	PASSOU
CT-WB-13	RF-01 CB	Obrigatório que também está no anyOf não é reportado em duplicidade	Chamar StructuralValidator().validate(entidade, schema) e inspecionar a lista de violações.	Schema de teste construído no próprio caso; validador sem estado.	required=[location]; anyOf location|address; entidade vazia	1 violação (location), não 2	Média	PASSOU
CT-WB-14	RF-01 CB	Tipo esperado desconhecido no schema é ignorado	Chamar StructuralValidator().validate(entidade, schema) e inspecionar a lista de violações.	Schema de teste construído no próprio caso; validador sem estado.	type='null' (não mapeado) e valor 1	Nenhuma violação	Média	PASSOU
CT-WB-15	RF-01 CB	Valor dict onde se espera string → found='object'	Chamar StructuralValidator().validate(entidade, schema) e inspecionar a lista de violações.	Schema de teste construído no próprio caso; validador sem estado.	x={'a': 1}	type_mismatch com found=object	Média	PASSOU
CT-WB-16	RF-01 CB	Valor lista onde se espera string → found='array'	Chamar StructuralValidator().validate(entidade, schema) e inspecionar a lista de violações.	Schema de teste construído no próprio caso; validador sem estado.	x=[1]	type_mismatch com found=array	Média	PASSOU
CT-WB-17	RF-01 CB	Valor de tipo exótico → found = nome da classe Python	Chamar StructuralValidator().validate(entidade, schema) e inspecionar a lista de violações.	Schema de teste construído no próprio caso; validador sem estado.	x=b'abc'	type_mismatch com found=bytes	Média	PASSOU
CT-WB-18	RF-01 CB	Booleano onde se espera string → found='boolean'	Chamar StructuralValidator().validate(entidade, schema) e inspecionar a lista de violações.	Schema de teste construído no próprio caso; validador sem estado.	x=True	type_mismatch com found=boolean	Média	PASSOU
CT-WB-19	RF-01 CB	anyOf parcialmente satisfeito: só o campo ausente é reportado	Chamar StructuralValidator().validate(entidade, schema) e inspecionar a lista de violações.	Schema de teste construído no próprio caso; validador sem estado.	a presente, b ausente; c ausente	1 violação: b (1ª alternativa)	Média	PASSOU

ID	Req. · Téc.	Nome	Descrição (passos)	Pré-condição	Dados de teste	Resultado esperado	Prior.	Status
CT-WB-20	RF-01 CB	anyOf satisfeito pela 2ª alternativa	Chamar StructuralValidator().validate(entidade, schema) e inspecionar a lista de violações.	Schema de teste construído no próprio caso; validador sem estado.	c presente	Nenhuma violação	Média	PASSOU
CT-CAR-01	RNF-01 CARGA	Smoke de concorrência no pytest	200 requisições concorrentes (GET /audits e POST /audits) via ASGITransport.	Nenhuma (in-process)	N=200; mistura 50% leitura / 50% escrita	0 respostas de erro; p95 < 2 s (teto de CI)	Alta	PASSOU
CT-CAR-02	RNF-01 CARGA	Carga nominal: 100 usuários por 45 s	Locust headless: 100 usuários, rampa 20/s, 45 s, mistura de endpoints da UI.	API-alvo (tests/carga/app_alvo.py) no ar, dependências externas simuladas.	100 usuários virtuais; espera 0,1–0,5 s	0 % de falhas e p95 < 500 ms em todos os endpoints	Alta	PASSOU
CT-CAR-03	RNF-02 CARGA	Estresse: 500 usuários por 45 s	Locust headless: 500 usuários, rampa 50/s, 45 s.	API-alvo (tests/carga/app_alvo.py) no ar, dependências externas simuladas.	500 usuários virtuais; espera 0,1–0,5 s	0 % de falhas (robustez); latência medida e registrada	Alta	PASSOU